FreeRTOS

The Need for RTOS

Task Scheduling

- Toggle an LED every 1 second.
- Toggle another LED every 2 second.
- Toggle another LED every 1.5 second.
- Toggle couple of other LED’s 3 seconds, 4.1 seconds, and 4.5 seconds
- Add breathing effect to an LED. (i.e., increase/decrease the intensity of LED over time)
- Add …

RTOS Basics

RTOS – How To ?

Define Tasks → Create Tasks → Call the Scheduler

```
task1()
{
    loop:
        toggle LED1
        dealy(1s)
}

task2()
{
    loop:
        Toggle LED2
        Delay(2s)
}

task3()
{
    loop:
        Toggle LED3
        Delay(3.2s)
}
```

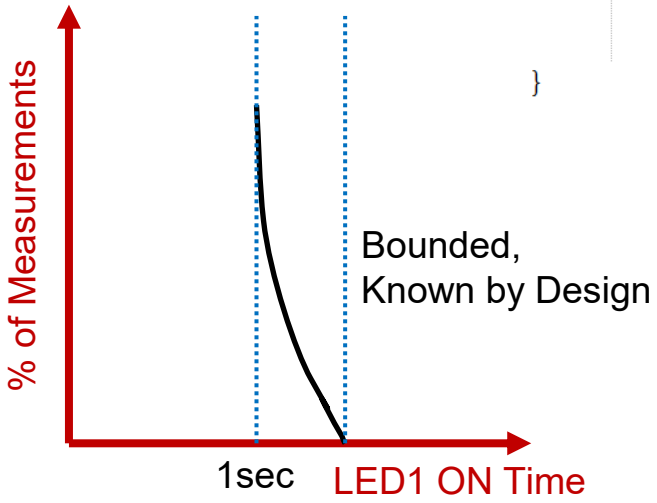
```
create(task1)
create(task2)
create(task3)

start scheduler
```

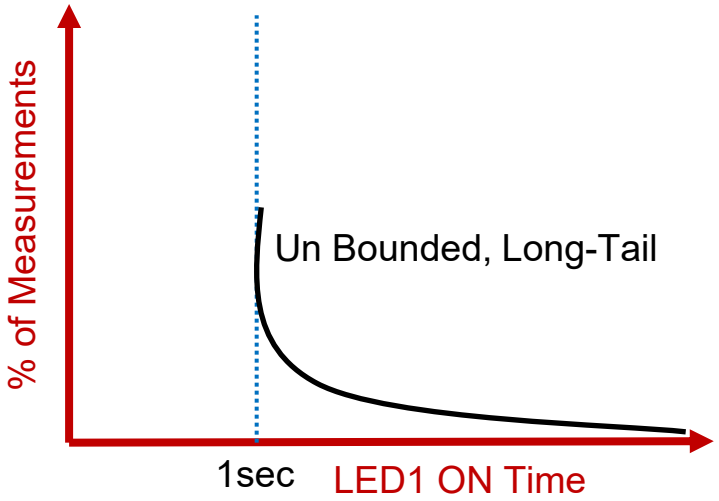
task-1, task-2, and task-3 toggles LED-1,2, and 3 every 1, 2 and 3.2 seconds, respectively.

Question: RTOS vs OS. What's the difference ?

RTOS vs. Generic OS

```
task1()
{
    loop:
        turn ON LED
        delay(1s)
        turn OFF LED
        delay(1s)
}
```





FreeRTOS Code

Define Tasks → Create Tasks → Call the Scheduler

```c
void led_task()
{
    while(1)
    {
        uint8 led_state = !led_Read();
        led_Write(led_state);
        vTaskDelay(250);
    }
}

void uart_task()
{
    uint16 counter = 1;
    char msg[32];

    while (1)
    {
        sprintf(msg, "Live %d\r\n", counter);
        UART_PutString(msg);

        counter++;

        vTaskDelay(pdMS_TO_TICKS(1000));
    }
}
```

```c
int main(void)
{
    CyGlobalIntEnable;

    UART_Start();
    UART_ClearTxBuffer();

    FreeRTOS_Start();

    xTaskCreate(led_task, "LED_Task", 200, NULL, 5, NULL);
    xTaskCreate(uart_task, "UART_Task", 200, NULL, 5, NULL);

    vTaskStartScheduler();

    while(1)
    {
    }
}
```

Above code runs two tasks, led_task and uart_task using FreeRTOS scheduler.

Scheduler

The scheduler uses priorities and time slicing (for tasks of equal priority) to run multiple tasks.

Scheduler Cycle

1. RTOS keeps a list of all tasks that are available to run.

2. It picks the **highest priority ready task**.

3. The CPU runs that task until:
   - It blocks e.g., calls Delay(), or waits for a semaphore, or an event, or
   - A higher-priority task becomes ready (e.g., triggered by an interrupt), or
   - Its allotted time-slice (e.g., 1ms) is over (only if there exist other tasks of same priority)

4. During task switching, the scheduler saves the current task's state and restores another's. Referred to as context switching.

E.g.,

Task	Priority
Task-H	3
Task-M	2
Task-L1	1
Task-L2	1

Assuming both preemption and time-slicing is enabled for the RTOS.

1. **At startup:**
   - All tasks are ready.
   - Scheduler checks priorities → runs **Task-H** first (priority 3).
2. **Task-H calls** `vTaskDelay(10)` → becomes *Blocked* for 10 ticks.
   - Scheduler now picks **Task-M** (next highest priority, 2).
3. **Task-M runs** and then also delays (`vTaskDelay(20)`).
   - Both H and M are now blocked → only **Task-L1** and **Task-L2** remain ready.
4. **Task-L1 and Task-L2** (both priority 1) share the CPU using **time slicing**.
   - Each runs for one tick (e.g., 1 ms) alternately:

```less
Time → L1 | L2 | L1 | L2 | ...
```

5. **At tick 10:**
   - Task-H becomes ready again → immediately **preempts** Task-L1/L2 (higher priority).
6. **At tick 20:**
   - Task-M becomes ready, but it runs **only when Task-H is blocked again**.

RTOS Keywords

Concept	Description
Task	Independent thread of execution
Scheduler	Decides which task runs next
Preemption	Higher-priority task can interrupt lower one
Context Switch	Saving/restoring task state during switch
Delay / Blocked State	Task waits; CPU given to others

Mutex

Application

```
task1()
{
    loop forever
    {
        arrADCBytes = read_ADC;
        uart_write(arrADCBytes)
        delay(50 ms);
    }
}

task2()
{
    loop forever
    {
        uart_write("Hello World");
        delay(1 ms);
    }
}
```

- Task-1 and Task-2 log data to the PC using a single UART hardware.
- Task-1 writes ADC output to the UART
- Task-2 writes a string output to the UART

Question – Any Issues ?

Note: Code is Generated by ChatGPT

Solution

```
create mutex xUART

task1()
{
    loop forever
    {
        arrADCBytes = read_ADC;

        take xUART
        uart_write(arrADCBytes)
        give xUART

        delay(50 ms);
    }
}
```

```
task2()
{
    loop forever
    {
        take xUART
        uart_write("Hello World");
        give xUART

        delay(1 ms);
    }
}
```

- Both tasks share the same UART peripheral, a critical resource.
- The mutex (xUART) ensures only one task uses UART at a time.
- Prevents interleaved or corrupted UART messages caused by concurrent access.

Semaphores

Application #1

```
task1()
{
    loop forever
    {
        arrDataBytes = read_SPI(30 bytes)
        delay(1.2ms)
    }
}

task2()
{
    loop forever
    {
        uart_write(arrDataBytes)
        delay(100ms)
    }
}
```

Task-1 updates an array with 30bytes from SPI

Task-2 transmits the array content via UART

Question – Any Issues ?

```
task1()
{
    loop forever
    {
        arrDataBytes = read_SPI(30 bytes)
        delay(1.2ms)
    }
}

task2()
{
    loop forever
    {
        uart_write(arrDataBytes)
        delay(100ms)
    }
}
```

⚠️ The Problem – Data Corruption

- Both tasks share the same buffer: arrDataBytes.
- While Task-1 writes new SPI data, the RTOS may preempt it and run Task-2.
- Task-2 then transmits a partially updated buffer over UART.
- Result: Mixed old and new data → inconsistent or corrupted output.
- Timing-dependent issue — may appear correct sometimes, corrupted at others.

Reference: ChatGPT

Solution

```
Create a binary semaphore xDataReady;

task1()    // SPI Reader Task
{
    loop forever
    {
        arrDataBytes = read_SPI(30 bytes);
        give xDataReady;
        delay(1.2 ms);
    }
}

task2()    // UART Sender Task
{
    loop forever
    {
        wait to take xDataReady;
        copy arrDataBytes to arrTemp;
        uart_write(arrTemp);
        delay(100 ms);
    }
}
```

xDataReady → Binary Semaphore

Semaphore here synchronize Task-1 and Task-2

Requirement: Operation “copy arrDataBytes to arrTemp” should be completed within 1.2ms. **Why ?**

You may also use mutex

Application #2

```
task1()
{
    ...
}

task2()
{
    ...
}

task3()
{
    ...
}
```

Question: Run task1, task2 and task3 at every 10ms.

How ?

Solution

```
Create a binary semaphore xTask3

master_task
{
    loop:
        give xTask1, xTask2, xTask3
        delay(10ms)
}

task1()
{
    loop:
        wait to take xTask1
        {
            ...
        }
}
```

```
task2()
{
    loop:
        wait to take xTask2
        {
            ...
        }
}

task3()
{
    loop:
        wait to take xTask3
        {
            ...
        }
}
```

Question: Run tasks in the following order, task1 → task2 → task3 → task1 …

Application #3

Question: Execute task1 when switch-1 is pressed and task2 when switch-2 is pressed. **How ?**

```
Create a binary semaphore xTask1
Create a binary semaphore xTask2

switch_1_isr
{
    give xTask1
}

switch_2_isr
{
    give xTask2
}
```

```
task1()
{
    loop:
        wait to take xTask1
        {
            ...
        }
}

task2()
{
    loop:
        wait to take xTask2
        {
            ...
        }
}
```

E.g., ISR Integration

Q: Toggle an LED when a switch is pressed

```c
CY_ISR (switch_isr_code)
{
    BaseType_t xHigherPriorityTaskWoken = pdFALSE;

    // Give semaphore from ISR
    xSemaphoreGiveFromISR(xButtonSemaphore, &xHigherPriorityTaskWoken);

    // Request context switch if needed
    portYIELD_FROM_ISR(xHigherPriorityTaskWoken);

    Pin_SW_ClearInterrupt();
}

void led_task()
{
    while(1)
    {
        /* Block indefinitely until semaphore is given */
        if (xSemaphoreTake(xButtonSemaphore, portMAX_DELAY) == pdTRUE)
        {
            uint8 led_state = !led_Read();
            led_Write(led_state);
        }

    }
}
```

```c
int main(void)
{
    CyGlobalIntEnable;

    isr_1_StartEx(switch_isr_code);

    FreeRTOS_Start();

    xButtonSemaphore = xSemaphoreCreateBinary();

    xTaskCreate(led_task, "LED_Task", 200, NULL, 5, NULL);

    vTaskStartScheduler();



    while(1)
    {
    }
}
```

Mutex vs. Semaphore

Core Differences

Feature	Mutex (Mutual Exclusion)	Semaphore
Purpose	Protect a *shared resource* (e.g., UART, I2C, buffer)	Signal or synchronize between *tasks or ISRs*
Ownership	Has ownership — **the task that takes it must release it**	No ownership — **any task can give or take**
Typical Usage	Prevent simultaneous access to critical section	Task synchronization or event signaling
Priority Inheritance	**Yes. Prevents priority inversion**	No

Priority Inversion - A high-priority task is forced to wait because a low-priority task holds a shared resource (e.g., a semaphore), while a medium-priority task keeps running and prevents the low-priority one from finishing and releasing it.

Priority Inheritance

Priority Inheritance Disabled. E.g., when using a semaphore

1. Task-L (Low-Priority) takes the semaphore (using UART).
2. Task-H (High-Priority) becomes ready and tries to take it → **blocked** (waiting).
3. Task-M (Medium-Priority) runs continuously (higher priority than Task-L).
4. Task-L can't finish and release UART because Task-M hogs the CPU.
5. Task-H waits forever → **priority inversion**! High-priority task is "stuck" behind a low one.

Priority Inheritance Enabled. E.g., when using a mutex

1. Task-L takes the mutex.
2. Task-H tries to take the same mutex → blocked.
3. RTOS **temporarily raises Task-L's priority** to match Task-H's priority.
4. Task-L runs immediately, finishes UART operation quickly, and releases the mutex.
5. Task-H resumes — no more inversion!

Queues

Application #1

```c
extern void FreeRTOS_Start();

static QueueHandle_t adcQueue;

typedef struct {
    uint16 index;      // sample index
    uint16 sample;     // 12-bit ADC sample (dummy)
} adc_msg_t;


void adc_task()
{
    adc_msg_t msg_adc;
    uint16 counter=0;

    while(1)
    {
        msg_adc.index = counter; counter++;
        msg_adc.sample = (uint16_t)(rand() & 0x0FFF);


        xQueueSend(adcQueue, &msg_adc, portMAX_DELAY);

        vTaskDelay(pdMS_TO_TICKS(100));
    }
}
```

```c
void uart_task()
{
    while(1)
    {
        adc_msg_t msg_rcv;
        char str[100];

        if (xQueueReceive(adcQueue, &msg_rcv, portMAX_DELAY) == pdTRUE)
        {
            sprintf(str, "ADC %d %d\r\n", msg_rcv.index, msg_rcv.sample);
            UART_PutString(str);
        }

        vTaskDelay(pdMS_TO_TICKS(100));
    }
}
```

```c
int main(void)
{
    CyGlobalIntEnable;

    UART_Start();
    UART_ClearTxBuffer();

    FreeRTOS_Start();

    /* Create queue to hold up to 32 adc_msg_t items */
    adcQueue = xQueueCreate(32, sizeof(adc_msg_t));

    xTaskCreate(adc_task, "ADC Task", 200, NULL, 5, NULL);
    xTaskCreate(uart_task, "UART_Task", 200, NULL, 5, NULL);

    vTaskStartScheduler();

    while(1)
    {
    }
}
```

What Next ?

Tasks

- Learn running FreeRTOS on PSoC-5LP
- Understand the importance of the arguments in xTaskCreate
- Learn – What is stack for a task in RTOS
- Task Priority – how to ?
- Sharing data between tasks. How to ?